

Relatório TP1

Serviços de Monitorização Urbana para One Health

GitHub: <https://github.com/Pedrom7/SistemasDistribuidos>

Elementos do Grupo:

Bruno Alves al80990

Liane Duarte al79012

Pedro Mendes al78140

Docentes:

Hugo Paredes | Cristiano Pendão | Tiago Pinto

1.Introdução

O conceito One Health reconhece que a saúde humana, animal e ambiental estão fortemente interligadas. Este trabalho tem como objetivo o desenvolvimento de um sistema distribuído que simula uma infraestrutura de monitorização ambiental urbana, composta por três entidades: o SENSOR, responsável pela recolha de dados ambientais; o GATEWAY, responsável pela validação, agregação e encaminhamento dos dados; e o SERVIDOR, responsável pelo armazenamento da informação. A comunicação entre as entidades é realizada através de sockets TCP em C#, com um protocolo textual definido pelo grupo. Os dados são persistidos numa base de dados MongoDB, escolhida por ser uma base de dados orientada a documentos, naturalmente compatível com o formato JSON das medições, e por permitir consultas analíticas sem pós-processamento. O servidor implementa ainda um mecanismo de fallback em ficheiros JSON locais: se o MongoDB estiver indisponível, os dados são escritos em disco sem interromper os gateways, retomando a escrita na base de dados assim que esta voltar a estar disponível.

2.Metodologia

O trabalho foi desenvolvido de forma incremental, seguindo as fases definidas no enunciado. Antes de qualquer implementação, o grupo reuniu para definir em conjunto o protocolo de comunicação mensagens, formatos, estados e casos de erro documentando todas as decisões e as respetivas justificações. Só após o protocolo estar estabilizado se avançou para a implementação.

A divisão de trabalho foi feita por entidade, sendo cada elemento responsável pela sua componente do início ao fim:

Elemento	Entidade	Responsabilidades principais
Bruno Alves (al80990)	GATEWAY	Validação de sensores no CSV, agregação em batch, encaminhamento para o servidor, gestão de estado, heartbeat e escalabilidade
Liane Duarte (al79012)	SERVIDOR	Receção e armazenamento de dados em MongoDB com fallback JSON, suporte a múltiplos gateways com threads e mutexes, validação de dados, reconstrução e armazenamento de vídeo em disco (.mp4).
Pedro Mendes (al78140)	SENSOR	Interface de texto, envio periódico de dados e heartbeat, stream de vídeo, manutenção e desconexão .

3.Protocolo de Comunicação

As mensagens seguem o formato textual TIPO | campo1 | campo2 | ..., com campos separados pelo carácter |. As respostas são sempre Ok ou ERRO | motivo. Foram definidos dois diálogos principais, cujos fluxos completos se encontram nos diagramas em anexo.

3.1 Diálogo SENSOR ↔ GATEWAY (porta 5001)

O registo é feito com `HELLO | sensorId | zona`, ao qual o gateway responde `Ok` se o sensor estiver no CSV, ou `ERRO | NOT_REGISTERED` caso contrário. Sensores em estado `manutencao` ou `desativado` são automaticamente restaurados para ativo ao ligar-se. O sensor envia `DATA | tipo | valor | timestamp` a cada 30 segundos (início 20s após arranque) para todos os tipos de dados que tem configurados, simulando um sensor físico real que mede em simultâneo tudo o que tem capacidade de medir. O `HEARTBEAT | sensorId | timestamp` é enviado a cada 60 segundos (início 30s após arranque), garantindo que nunca coincide com o ciclo de dados. O gateway deteta ausência de comunicação superior a 120 segundos e atualiza o estado do sensor para `desativado` no CSV. Para stream de vídeo, o sensor envia `STREAM_START`, seguido de frames em base64 via `STREAM_DATA` sem ACK por frame, e termina com `STREAM_END`. O gateway acumula todos os frames em memória e encaminha-os ao servidor via `VIDEO_FORWARD` após o `STREAM_END` é o servidor que processa o vídeo. A manutenção é comunicada pelo sensor através de `MAINTENANCE | sensorId`, sinalizando uma interrupção temporária planeada. A desconexão definitiva é feita com `DISCONNECT | sensorId`. O gateway responde `HEARTBEAT_OK` ao heartbeat, `MAINTENANCE_OK` à manutenção (encerrando a ligação de seguida), e `ERRO | TIPO_NAO_SUPORTADO` se o tipo de um DATA não estiver na lista de tipos suportados para esse sensor no CSV. O contador de timeout de 120 segundos só começa 30 segundos após a ligação, o tempo do primeiro heartbeat, para que o silêncio inicial não seja interpretado como falha. Em caso de falha de rede, o sensor acumula medições num buffer local em fila (máximo 20); quando a ligação é restabelecida, reenvia por ordem de chegada antes de retomar o envio normal.

3.2 Diálogo GATEWAY ↔ SERVIDOR (porta 5000)

A ligação é permanente, estabelecida uma única vez no arranque com `HELLO | gatewayId`, eliminando o overhead de múltiplos handshakes TCP. O gateway acumula medições em buffer local e envia ao servidor em batch de 5, numa única mensagem `FORWARD` com todas as medições serializadas: `FORWARD | sensorId | zona | ts, tipo, valor; ts, tipo, valor; ...`, as medições são separadas por `;` e cada uma tem o formato `timestamp, tipo, valor`. As cópias locais são apagadas após confirmação `Ok` do servidor; em caso de `ERRO`, o batch é mantido em buffer e recolocado para reenvio, sem perda de dados. Para vídeo, o gateway encaminha os frames acumulados via `VIDEO_FORWARD | sensorId | zona | timestamp | parte | totalPartes | frames`, onde os frames são separados por `;;` (e não por `;`) para não colidir com o separador de medições do `FORWARD`. Vídeos grandes são divididos em partes de até 100 frames. O servidor recebe, processa e armazena o resultado. A ligação ao servidor é encerrada com `DISCONNECT | gatewayId` quando o gateway termina.

4. Opções de Implementação

A implementação foi realizada em C# utilizando sockets TCP através das classes `TcpListener` e `TcpClient` da framework .NET, com comunicação baseada em `StreamReader` e `StreamWriter`. Esta escolha permite controlo direto sobre a ligação e é adequada ao protocolo textual definido.

O **SENSOR** implementa uma interface de linha de comandos que permite ao utilizador simular o envio de medições ambientais, heartbeat, streams de vídeo real, manutenção e desconexão. O envio de DATA e de HEARTBEAT é feito em threads separadas, garantindo que os dois ciclos nunca bloqueiam nem coincidem. Em cada ciclo de 30 segundos, o sensor envia medições para todos os tipos de dados que tem configurados por argumento de arranque, simulando leituras simultâneas de múltiplos sensores físicos. Os valores são gerados de forma realista por tipo (TEMP entre 10–35 °C, PM2.5 entre 5–155 µg/m³, etc.). Ao receber `REDIRECT | ip | porta`, o sensor fecha a ligação atual e reconecta automaticamente ao novo gateway, sem intervenção do utilizador. Em caso de falha de rede, o sensor mantém um buffer local em fila (máximo 20 medições); quando a ligação é restabelecida, as medições acumuladas são reenviadas por ordem

de chegada antes de retomar o envio normal. O heartbeat não é guardado em buffer por ser um sinal de presença em tempo real. Os frames de vídeo são enviados em base64 sem quebras de linha, garantindo que cada frame é transmitido como uma única mensagem TCP.

O GATEWAY mantém um ficheiro CSV local com o registo de todos os sensores autorizados, os seus estados (ativo, manutencao, desativado), zonas e tipos de dados suportados. Para cada ligação de sensor é criada uma thread dedicada, permitindo atendimento concorrente. Três mutexes protegem os recursos partilhados: lockCsv para leituras e escritas no ficheiro CSV, lockServidor para a ligação TCP ao servidor, e lockHeartbeat para o dicionário de timestamps de última atividade. O gateway mantém uma ligação permanente ao servidor, estabelecida no arranque. As medições são acumuladas em batch de 5 e enviadas numa única mensagem FORWARD ao servidor, com cópia local de segurança por medição apagada após confirmação, ou recolocada em fila em caso de erro. Os frames de vídeo são acumulados em memória durante o stream e encaminhados ao servidor em partes de até 100 frames após o STREAM_END. Quando o número de sensores ligados atinge o limite configurável, o gateway serializa o CSV com ;; como separador de linhas e sincroniza-o com o novo gateway via SYNC_CONFIG antes de redirecionar o sensor com REDIRECT.

O SERVIDOR aceita ligações de múltiplos gateways em simultâneo, criando uma thread dedicada por gateway. Os dados recebidos via FORWARD são desserializados (split por ; e depois por ,), validados (timestamp e valor numérico) e inseridos em batch na base de dados **MongoDB** (base de dados monitorizacao_urbana, coleção medicoes), com os campos sensorId, zona, tipo, valor (numérico) e timestamp. Se o MongoDB estiver indisponível, o servidor escreve automaticamente em ficheiros **JSON locais** em dados/{zona}/{sensor}/{tipo}.json como fallback, respondendo sempre Ok ao gateway enquanto pelo menos um dos mecanismos estiver disponível, só responde ERRO | STORAGE_ERROR se ambos falharem. Os frames de vídeo recebidos via VIDEO_FORWARD são acumulados em memória por (sensorId, timestamp) e, quando a última parte chega, o vídeo é **reconstruído** a partir dos chunks em base64 e gravado como ficheiro .mp4 real em dados_video/{zona}/{sensor}/. Os metadados do vídeo (sensorId, zona, timestamp, ficheiro, caminho) são inseridos na coleção videos do MongoDB, ou escritos num ficheiro.json ao lado do .mp4 em caso de fallback. Esta separação entre metadados em base de dados e ficheiros binários em disco segue o padrão utilizado em sistemas reais de media (S3/disco para blobs, base de dados para metadados estruturados). O acesso concorrente é gerido pelo driver MongoDB, que é thread-safe. Para a escrita dos ficheiros .mp4 em disco, mantém-se um mutex por ficheiro, permitindo que vídeos de sensores diferentes sejam gravados em paralelo sem bloqueio.

4.Conclusão

Este trabalho permitiu desenvolver um sistema distribuído de monitorização ambiental urbana com comunicação TCP, protocolo textual próprio e mecanismos de tolerância a falhas. Foram implementados buffering local, reconexão automática, batching de mensagens, deteção de timeout e escalabilidade horizontal via REDIRECT e SYNC_CONFIG. O trabalho cumpre os requisitos definidos no enunciado e constituiu uma experiência prática relevante no desenvolvimento de sistemas distribuídos reais.

5.Anexos

Diagrama 1 - Registo (HELLO)



Diagrama 2 - Envio de dados (DATA)

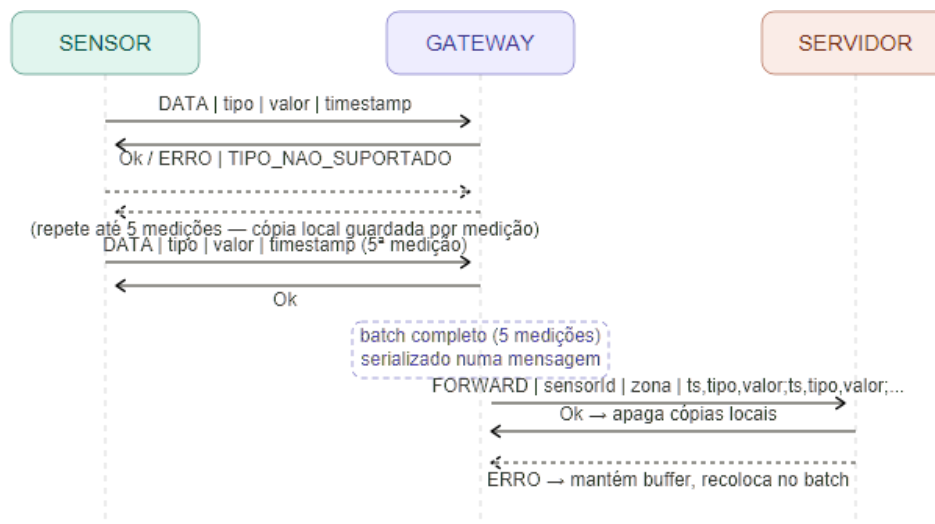


Diagrama 3 - Heartbeat

Diagrama 3 — Heartbeat e timeout

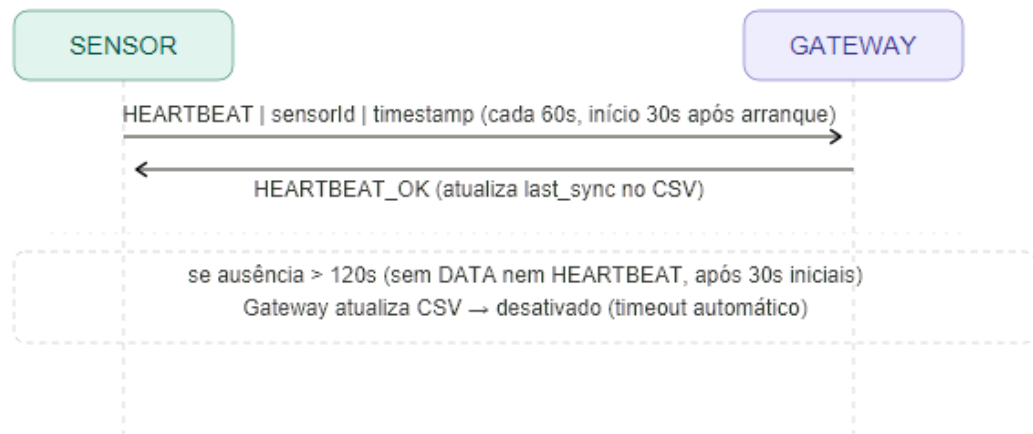


Diagrama 4 - Stream de vídeo

Diagrama 4 — Stream de vídeo

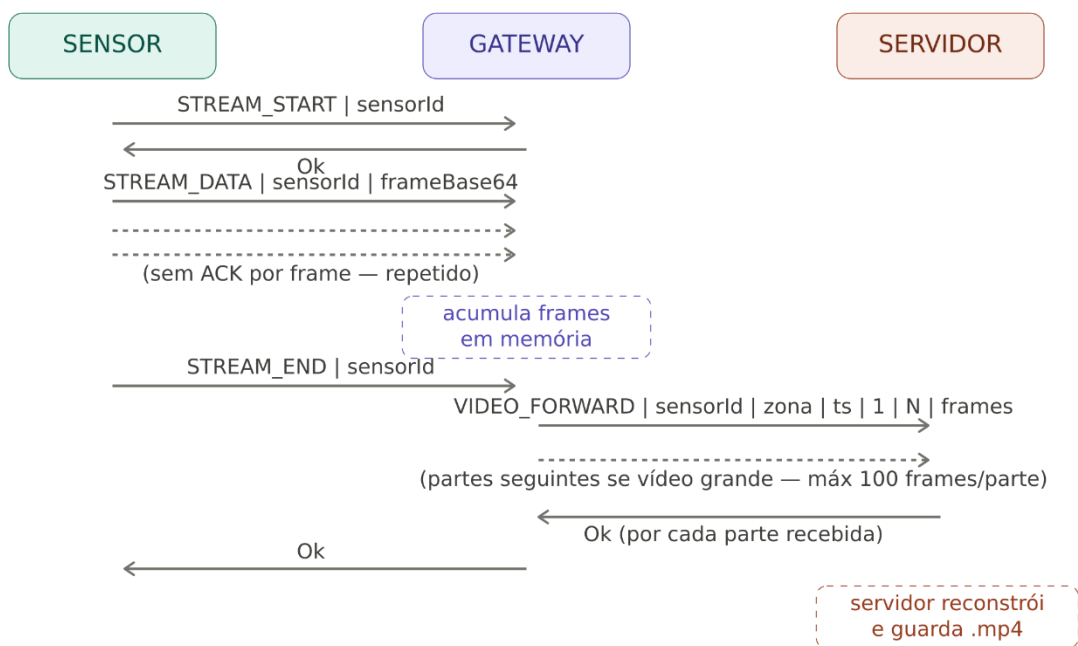


Diagrama 5 - Manutenção e desconexão

Diagrama 5 — Manutenção e desconexão

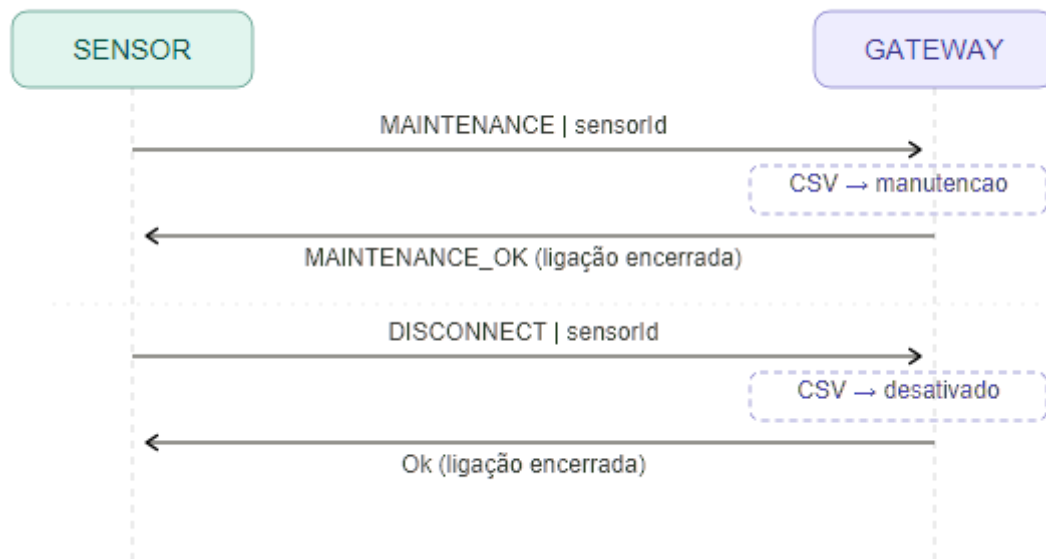
**Diagrama 6 - REDIRECT e SYNC_CONFIG**

Diagrama 6 — Escalabilidade: REDIRECT e SYNC_CONFIG

